

100 Preguntas — Entrevista Frontend React

React · Next.js · Redux · Performance · TypeScript · Testing · A11y · Jul 2026

Índice: React base (1–15) · Hooks (16–30) · Patrones (31–40) · Next.js (41–55) · Estado/Redux (56–65) · Performance (66–80) · TypeScript (81–85) · Testing (86–90) · A11y/Web (91–95) · Sistema (96–100) · **Extra:** Errores Krunchbox · useMemo/useCallback

1. REACT — FUNDAMENTOS (1–15)

1. ¿Qué es React y por qué usarlo?

Librería UI basada en componentes declarativos. Virtual DOM + reconciliación minimizan manipulación directa del DOM. Ecosistema maduro, composición, SSR con Next.

2. ¿Virtual DOM vs DOM real?

React mantiene árbol JS en memoria. Tras un update compara (diff) y aplica solo cambios mínimos al DOM real (commit). No es más rápido siempre, pero simplifica UI compleja.

3. ¿Qué es JSX?

Sintaxis que parece HTML en JS. Se transpila a `React.createElement`. Una expresión por nodo; `className` en vez de `class`.

4. ¿Element vs Component?

Elemento: objeto ligero que describe qué renderizar. Componente: función/clase que devuelve elementos. `<Button />` invoca la función del componente.

5. ¿Props vs State?

Props: entrada inmutable del padre. State: datos internos mutables vía `setState` / `useState`. Cambios en ambos disparan re-render.

6. ¿Unidireccionalidad de datos?

Datos bajan por props; eventos suben por callbacks. Facilita depuración y flujo predecible vs two-way binding clásico.

7. ¿Qué es reconciliación?

Algoritmo que compara árbol anterior y nuevo. Misma key + mismo tipo → actualiza props. Tipo distinto → desmonta y monta. Keys estables en listas evitan bugs.

8. ¿Por qué no usar index como key?

Reordenar/insertar/borrar reutiliza nodos incorrectos → estado mezclado, inputs corruptos, animaciones rotas. Usa ID estable del dato.

9. ¿Controlled vs Uncontrolled?

Controlled: React controla `value` + `onChange`. Uncontrolled: DOM guarda valor vía `ref`. RHF mezcla ambos para performance en forms grandes.

10. ¿Lifting state up?

Subir estado al ancestro común cuando dos hermanos deben compartirlo. Evita duplicar fuente de verdad.

11. ¿Composition vs Inheritance?

React favorece composición (`children`, render props, slots). No herencia de clases para extender UI.

12. ¿Strict Mode en dev?

Monta/desmonta y re-ejecuta efectos dos veces en dev para detectar side effects no idempotentes. No afecta producción.

13. ¿Error Boundaries?

Solo class components con `getDerivedStateFromError` / `componentDidCatch` capturan errores de render/hijos. No atrapan event handlers ni async. En App Router: `error.tsx`.

14. ¿Portals?

`createPortal` renderiza hijos fuera del DOM padre (modales, tooltips) manteniendo contexto React y event bubbling.

15. ¿React 19 novedades clave?

Actions, `useActionState`, `useOptimistic`, `ref` como prop, mejor hidratación, `use` para promises/context, document metadata en componentes.

2. HOOKS (16–30)

16. Reglas de los Hooks

Solo en top-level (no en if/loops). Solo en componentes funcionales o custom hooks. Garantizan orden estable entre renders.

17. ¿useState batching?

Múltiples `setState` en un evento se agrupan en un solo re-render. React 18+: batching también en promises/timeouts.

18. ¿useEffect vs useLayoutEffect?

`useEffect`: tras paint (no bloquea). `useLayoutEffect`: tras DOM update, antes de paint — mediciones DOM, evitar flicker. En SSR ambos no corren en servidor.

19. Dependencias de useEffect

Array con todo valor externo usado. Omitir = cada render. `[]` = solo mount. ESLint `exhaustive-deps` ayuda; no silenciar sin razón.

20. Cleanup en useEffect

Return function para abortar fetch, quitar listeners, `clearInterval`. Evita memory leaks y race conditions.

21. ¿useRef vs useState?

`useRef`: mutable sin re-render (DOM refs, timers, flags). `useState`: cambio visible en UI. No leer/escribir `ref.current` durante render si afecta output.

22. ¿useMemo cuándo?

Cálculo caro o referencia estable para `memo` /deps. No por defecto en todo — coste de comparación. Perfil primero.

24. ¿useReducer vs useState?

Reducer para estado complejo, transiciones explícitas, testing del reducer puro. Misma API de dispatch que Redux.

26. ¿useContext gotchas?

Cualquier cambio de valor re-renderiza todos los consumidores. Split contexts (theme vs user). No para datos de alta frecuencia.

28. ¿useTransition / useDeferredValue?

Marcan updates como no urgentes. UI responsive mientras render pesado (filtros, listas). Concurrent features de React 18+.

30. ¿React Compiler (Forget)?

Compilador que memoiza automáticamente. Reduce necesidad manual de `memo` / `useMemo`. Aún adoptando; entender reconciliación sigue siendo clave.

23. ¿useCallback cuándo?

Estabiliza función pasada a hijo `memo` o en deps de effect. Innecesario si el hijo no está memoizado.

25. ¿Custom hooks?

Funciones `use*` que encapsulan lógica con hooks. Comparten comportamiento sin HOC/render props. Ej: `useDebounce`, `useMediaQuery`.

27. ¿useId?

IDs únicos estables SSR/cliente para `htmlFor` / `aria-*`. Evita colisiones en hidratación.

29. Stale closures en hooks

Callback captura valores viejos si deps incorrectas. Solución: deps correctas, `useRef` para valor actual, o funcional update `setCount(c => c+1)`.

3. PATRONES Y ARQUITECTURA (31–40)

31. Container/Presentational

Separar lógica/datos (container) de UI pura (presentational). Hoy: custom hooks + componentes tontos.

33. Render Props vs HOC

Render prop: función como child. HOC: envuelve componente. Ambos comparten lógica; hooks los reemplazan en muchos casos. Cuidado wrapper hell.

35. Smart vs Dumb components

Smart: conectan datos/estado. Dumb: solo props. Facilita Storybook, tests y reutilización.

37. Colocation

Mantener estilos, tests y lógica cerca del componente que usan. Menos saltos entre archivos.

39. Anti-patterns comunes

`useEffect` para derivar estado (usar variable/cálculo), context gigante, index keys, mutar state directamente, fetch sin cleanup/abort.

32. Compound Components

API flexible: `<Select><Select.Option /></Select>` comparten estado interno vía context. Radix, Headless UI.

34. Feature-Sliced / por dominio

Carpets por feature (`policies/`, `auth/`) con `ui/hooks/api/types`. Escala mejor que solo por tipo técnico.

36. Atomic Design

Átomos → moléculas → organismos → templates → pages. Útil en design systems; no dogma de carpetas.

38. Prop drilling — soluciones

Composición (`children`), context acotado, state library (Zustand/Redux), React Query para server state.

40. Suspense para qué

Declarar UI de loading mientras hijo async (lazy, RSC, data con `use`). Boundaries anidados para granularidad.

4. NEXT.JS — APP ROUTER (41–55)

41. Pages vs App Router

App Router (13+): layouts anidados, RSC por defecto, streaming, `loading.tsx`. Pages: `getServerSideProps` / `getStaticProps` — legacy pero válido.

43. ¿Cuándo 'use client'?

`useState/useEffect`, `onClick`, context providers, librerías solo browser. Mantener boundary lo más bajo posible.

45. Route Handlers

`app/api/.../route.ts` exporta GET/POST. Sustituto de API routes Pages. Edge o Node runtime según necesidad.

47. dynamic() import

Code splitting con loading opcional. `ssr: false` solo client. Reduce bundle inicial.

42. Server vs Client Components

Server: async, acceso DB/secrets, cero JS al cliente. Client: `'use client'`, hooks, browser APIs. Componer: server padre + client hijo.

44. Fetch en Server Components

`await fetch` en componente server; Next dedupe y cache por defecto. `cache: 'no-store'` dinámico; `revalidate: 60` ISR.

46. Middleware Next.js

`middleware.ts` en edge: auth redirect, locale, headers. Corre antes del render.

48. Metadata API

`export const metadata` o `generateMetadata` para SEO, OG tags. Estático o dinámico por ruta.

49. Server Actions

Funciones server invocables desde client forms. Mutaciones sin API REST explícita. Validar siempre en servidor.

51. ISR vs SSG vs SSR

SSG: build time. ISR: SSG + revalidate. SSR: cada request (o cache CDN). Elegir por frescura vs carga.

53. next/image

Lazy load, sizes responsive, WebP/AVIF, evita CLS con width/height. Dominios en `next.config`.

55. Caching Next 15

Fetch cache, Full Route Cache, Router Cache cliente.
`revalidatePath` / `revalidateTag` invalidación on-demand.

50. Hydration mismatch

HTML servidor $\neq$ primer render cliente (`Date.now`, `window`, `random`). Usar `useEffect`, `suppressHydrationWarning` puntual, o render solo client.

52. Parallel & Intercepting Routes

`@modal` `slots`, `(.)photo` `intercept` — modales con URL shareable sin perder contexto de lista.

54. Env variables

`NEXT_PUBLIC_*` expuesto al browser. Resto solo server. Nunca secrets en client bundle.

5. ESTADO GLOBAL — REDUX / ALTERNATIVAS (56–65)

56. ¿Cuándo Redux?

Estado global complejo, muchas fuentes de updates, time-travel debug, middleware (sagas). Overkill para UI local o server state.

58. Flujo Redux

UI dispatch action $\rightarrow$ reducer puro $\rightarrow$ nuevo state $\rightarrow$ store $\rightarrow$ selectors $\rightarrow$ re-render suscriptores.

60. RTK Query

Cache server state en Redux: endpoints, tags, invalidación. Alternativa a React Query si ya usas Redux.

62. Redux vs Context

Context: pocos valores, simple. Redux: updates frecuentes, devtools, middleware, normalización. Context no escala con muchos consumidores.

64. Zustand / Jotai

Stores ligeros sin boilerplate. Zustand: store único. Jotai: átomos. Bueno para estado medio sin Redux.

57. Redux Toolkit ventajas

`createSlice`, `Immer` `immutable`, `configureStore`, menos boilerplate. Estándar actual vs Redux clásico.

59. createAsyncThunk

Thunk con `pending/fulfilled/rejected`. Integra `loading/error` en slice sin escribir máquina manual.

61. Selectores memoizados

`createSelector` (`reselect`) evita recalcular derivados y reduce re-renders en `useSelector`.

63. Server state vs Client state

Server: React Query/SWR/RTK Query. Client: UI, modals, wizard step. No duplicar API data en Redux sin necesidad.

65. Normalización de estado

Entidades por ID en objetos (`byId`, `allIds`). Evita duplicados y facilita updates parciales (patrón Redux en listas).

6. PERFORMANCE Y OPTIMIZACIÓN (66–80)

66. Cómo medir performance

React Profiler, Lighthouse, Web Vitals (LCP, INP, CLS). Chrome Performance tab. Medir antes de optimizar.

68. Virtualización de listas

Render solo filas visibles (`react-window`, `tanstack-virtual`). Esencial con 10k+ items. Pluvia/ag-Grid patrón enterprise.

70. Web Vitals — LCP

Largest Contentful Paint $<2.5s$. Optimizar hero image, fonts, SSR, CDN, quitar render-blocking JS.

72. CLS

Cumulative Layout Shift: reservar espacio imágenes/fonts, no insertar banners encima de contenido, skeletons con altura fija.

74. Bundle analysis

`@next/bundle-analyzer`, `webpack-bundle-analyzer`. Detectar librerías pesadas, `tree-shaking`, imports específicos.

76. Prefetch y preload

Next `Link` `prefetch` rutas. `rel=preload` fonts/critical assets. React Query `prefetchQuery` en hover.

67. React.memo

Evita re-render si props shallow-equal. Útil en listas/items pesados. Comparador custom si props complejas.

69. Code splitting

`React.lazy` + `Suspense`, `dynamic()` en Next, route-based splitting automático. Bundle por ruta/feature.

71. INP (Interaction to Next Paint)

Latencia de interacción. Reducir JS largo, usar transitions, workers, dividir tareas, evitar re-renders masivos en input.

73. Debounce vs Throttle

Debounce: ejecuta tras pausa (búsqueda). Throttle: máximo N veces por intervalo (scroll). Ambos reducen trabajo.

75. Tree shaking

Eliminar código muerto en build. Import named/ESM; evitar barrel files enormes que impiden shake.

77. Optimistic updates

UI actualiza antes de respuesta server; rollback si falla. React Query `onMutate`, Redux `optimistic reducer`, `useOptimistic`.

78. Memoización excesiva

Anti-pattern: memo en todo. Coste comparación + complejidad.
Optimizar hot paths identificados con profiler.

80. Workers / Off-main-thread

Comlink, Web Workers para parse CSV, cripto, cálculos pesados sin bloquear INP.

79. SSR vs CSR trade-offs

SSR: SEO, FCP rápido, más carga servidor. CSR: SPA simple, TTI puede tardar. Híbrido RSC es el sweet spot Next.

7. TYPESCRIPT + REACT (81–85)

81. Tipar props de componente

`type Props = { title: string; onSave?: () => void } . React.FC`
ya no recomendado; función explícita + `children?: ReactNode`.

83. Event types

`ChangeEvent<HTMLInputElement>`, `MouseEvent<HTMLButtonElement>`.
Inferir del handler evita errores.

85. Tipar hooks custom

Return type explícito o inferido. Genéricos en hooks de fetch:
`useApi<Policy>(url)`. Never any en boundaries públicos.

82. Generics en componentes

`function List<T>({ items, render }: { items: T[]; render: (i: T) => ReactNode })` — reusable type-safe.

84. Utility types útiles

`Pick`, `Omit`, `Partial`, `Record`, `satisfies` para props derivadas de API types.

8. TESTING (86–90)

86. Pirámide de testing front

Muchos unit (dominio, utils), integración (RTL + MSW), pocos E2E (Playwright). No snapshot masivo de árboles.

88. Mocking fetch / API

MSW intercepta red; `vi.fn()` en Vitest para unit. Probar loading/error/success paths.

90. Qué testear en un hook

`renderHook` + `act`: estado inicial, transiciones, cleanup, error. Extraer lógica del componente facilita coverage.

87. Testing Library filosofía

Testear como usuario: roles, labels, text. `getByRole` > `querySelector`. Evitar detalles de implementación.

89. Vitest vs Jest

API compatible, más rápido con Vite/ESM, `vi.mock` nativo. Misma mentalidad describe/it/expect.

9. ACCESIBILIDAD Y WEB (91–95)

91. ARIA cuándo usar

Primero HTML semántico (`button`, `nav`, `label`). ARIA solo si no hay elemento nativo. `aria-live` para toasts dinámicos.

93. WCAG niveles

A, AA (estándar legal EU), AAA. Contraste 4.5:1 texto normal, teclado completo, textos alternativos.

95. CSP y XSS

Content-Security-Policy limita scripts. Escapar HTML user input; evitar `dangerouslySetInnerHTML`. DOMPurify si rich text.

92. Focus management en modales

Trap focus, `aria-modal`, devolver focus al trigger al cerrar. Radix/Headless lo implementan.

94. CORS en frontend

Browser bloquea respuestas cross-origin sin headers CORS del server. Proxy Next rewrites o API same-origin. No "arreglable" solo en client.

10. DISEÑO DE SISTEMA Y BEHAVIORAL (96–100)

96. Diseña un dashboard de pólizas

RSC lista inicial + client grid virtualizado. React Query cache + paginación server. Filtros debounced. Error boundaries por widget. Auth en middleware.

98. Convivencia con legacy jQuery

Islands React en mounts puntuales, contrato de eventos custom, no doble manipulación DOM, migración incremental por ruta.

97. Manejo de errores en producción

Boundaries UI, toast + log Sentry, retry con backoff, fallbacks degradados, nunca pantalla blanca sin mensaje.

99. Cómo priorizas deuda técnica

Impacto × esfuerzo × riesgo. Quick wins en hot paths, tests en dominio crítico, ADRs para decisiones, no big-bang rewrite.

100. Pregunta trampa: ¿React es un framework?

Es librería de UI (Vista). Next.js/Remix sí son frameworks full-stack. React no trae router, data layer ni convenciones de proyecto — tú eliges el stack.

FILOSOFÍA EN B2B ENTERPRISE

En Pluvia (analytics retail, 100k+ filas) un fallo en un widget **no tumba el dashboard**. Errores predecibles, mensajes accionables, sin PII en logs cliente. Validar en boundary, mostrar en UI, reportar en servidor.

Capa	Responsabilidad	Herramienta
1. Boundary	Validar request/response API antes de tocar UI	Zod safeParse → 400 con fieldErrors
2. Servicio	Result tipado, no throw genérico	{ ok: true, data } { ok: false, error, code }
3. Fetch hook	loading · error · empty · success	React Query / custom hook con AbortController
4. Widget UI	Fallback local si falla un chart/grid	Error Boundary por widget + retry
5. Ruta	Crash de segmento Next	error.tsx + reset()
6. Observabilidad	Stack trace, digest, contexto técnico	Sentry server; cliente sin emails/IDs sensibles

ZOD EN BOUNDARY (PATRÓN)

```
// schemas/policy.ts
export const policyQuerySchema = z.object({
  retailerId: z.string().uuid(),
  from: z.coerce.date(),
  to: z.coerce.date(),
})

export function parsePolicyQuery(raw: unknown) {
  const parsed = policyQuerySchema.safeParse(raw)
  if (!parsed.success) {
    return { success: false as const,
      error: parsed.error.flatten().fieldErrors }
  }
  return { success: true as const, data: parsed.data }
}

// route.ts → 400 si !success
// hook → nunca asumir shape del API sin validar
```

UI POR TIPO DE ERROR

Tipo	UX
400 validación	Inline en form / resaltar campo
401 / 403	Redirect login o “sin permiso”
404	Empty state + link volver
5xx / red	Toast + botón Reintentar
Render crash	Boundary: mensaje + retry widget
Grid SSRM	ag-Grid overlay error + refetch datasource

HOOK + GRID (PLUVIA-STYLE)

```
function useGridData(filters: Filters) {
  const [state, setState] = useState<
    'idle' | 'loading' | 'error' | 'success'
  >('idle')
  const [error, setError] = useState<AppError | null>(null)

  useEffect(() => {
    const ctrl = new AbortController()
    setState('loading')
    fetchGrid(filters, { signal: ctrl.signal })
      .then(res => {
        if (!res.ok) throw toAppError(res) // mapea status → code
        return res.json()
      })
      .then(json => {
        const parsed = gridResponseSchema.safeParse(json)
        if (!parsed.success) throw new AppError('INVALID_RESPONSE')
        setState('success')
        return parsed.data
      })
      .catch(err => {
        if (err.name === 'AbortError') return // filtro nuevo, ignorar
        setError(err)
        setState('error')
      })
    return () => ctrl.abort()
  }, [filters])

  return { state, error, retry: () => ... }
}

// Retry: solo GET / lecturas idempotentes
// POST guardar: no auto-retry; confirmar usuario
```

ERROR BOUNDARY POR WIDGET

```
// Dashboard.tsx
<ErrorBoundary fallback={<ChartError onRetry />}>
  <SalesChart data={data} />
</ErrorBoundary>
<ErrorBoundary fallback={<GridError />}>
  <PluviaGrid /> { /* ag-Grid aislado */ }
</ErrorBoundary>

// Next App Router: error.tsx por segmento
// reset() re-renderiza sin full reload
```

QUÉ NO HACER (ENTREVISTA)

- `catch {}` vacío o solo `console.log`
- Mostrar stack trace al usuario final
- Loguear PII (email, policyId) en Sentry cliente
- Retry automático en POST de pago/guardado
- Un Error Boundary global sin fallbacks por sección
- Confiar en shape del API sin Zod/runtime check

RESPUESTA ENTREVISTA (30 S)

“Capas: Zod en el boundary del API, Result tipado en servicios, estados loading/error/empty en cada widget. Error Boundaries por chart y grid para que un fallo no tire el dashboard. Retry solo en lecturas idempotentes con AbortController en filtros. Logs a Sentry en servidor; en cliente mensajes humanos sin datos sensibles. En Pluvia con ag-Grid SSRM usamos overlay de error y refetch del datasource.”

EXTRA B — USEMEMO / USECALLBACK: CUÁNDO SÍ Y CUÁNDO NO

REGLA DE ORO

Medir primero (React Profiler). Memoizar tiene coste: comparar deps + memoria. En Krunchbox/Pluvia solo optimizamos hot paths: celdas de grid, columnDefs, handlers pasados a memo children, filtros sobre datasets grandes.

USEMEMO — SÍ USAR

Caso	Por qué
Cálculo caro	Filtrar/ordenar 10k–100k filas, agregaciones
Referencia estable	Objeto/array pasado a hijo <code>memo()</code> o <code>useEffect</code> deps
columnDefs ag-Grid	Evitar recrear defs cada render → grid no parpadea
Context value	<code>useMemo(() => ({ user, theme })), [user, theme])</code>
Derivar de props	<code>visibleRows = rows.filter(...)</code> si filter es O(n) grande

USEMEMO — NO USAR

Caso	Alternativa
Suma, map pequeño (<100 items)	Cálculo directo en render
“Por si acaso” sin profiler	Nada — YAGNI
Primitivos ($n * 2$)	Variable normal
Estado que debería ser state	<code>useState</code> + <code>effect</code> o <code>derived</code> en evento

USECALLBACK — SÍ USAR

Caso	Por qué
Hijo <code>React.memo</code>	Misma ref de función → skip re-render
Deps de <code>useEffect</code>	Evitar re-run infinito del effect
Virtual list / grid cell	Renderer estable por fila (miles de nodos)
Callback a lib externa	ag-Grid <code>onFilterChanged</code> , DnD sensors
Custom hook API	Exponer <code>onSave</code> estable al consumidor

USECALLBACK — NO USAR

Caso	Por qué
Hijo sin memo	Re-renderiza igual; callback no ayuda
Handler en botón suelto	Coste re-render negligible
Dentro de listas sin memo item	N funciones nuevas igualmente
Deps cambian cada render	<code>useCallback(fn, [obj])</code> inútil si obj es nuevo

EJEMPLO PLUVIA — GRID + FILTROS

```
const columnDefs = useMemo(() => buildColumns(locale, permissions), [
  locale, permissions,
])

const onFilterChanged = useCallback((model: FilterModel) => {
  setFilters(model) // dispara SSRM refetch
}, [])

const CellRenderer = useMemo(
  () => memo(function Cell({ value }: { value: number }) {
    return <span>{formatCurrency(value)}</span>
  }),
  [locale],
)

// ❌ useCallback en cada render de 500 filas sin memo
// ✅ memo en cell + columnDefs estables + debounce filtros
```

ÁRBOL DE DECISIÓN RÁPIDO

```
¿El cálculo tarda >1ms en Profiler?
  SÍ → useMemo
  NO → cálculo inline

¿El hijo está envuelto en memo()?
  SÍ → useCallback para props función
  NO → handler inline OK

¿La referencia está en deps de useEffect?
  SÍ → useCallback o mover lógica dentro del effect
  NO → no necesario

¿React Compiler / Forget activo en proyecto?
  SÍ → menos memo manual; seguir midiendo
  NO → optimizar solo hot paths identificados
```

RESPUESTA ENTREVISTA (20 S)

“No memoizo por defecto. Uso useMemo para cálculos caros o referencias estables en grids y context; useCallback cuando paso funciones a hijos memoizados o como deps de effects. En Pluvia perfilamos con Profiler: columnDefs, cell renderers y filtros sobre 100k filas. El resto del árbol se optimiza con virtualización y state colocado, no con memo en todo.”